

L'Ereditarietà

L'**ereditarietà** è un concetto fondamentale dell'**OOD/OOP**. È possibile in un linguaggio **OO** creare nuove classi a partire da classi già esistenti ereditando le caratteristiche (**attributi** e/o **metodi**), aggiungendone di nuove o ridefinendone alcune. L'ereditarietà è finalizzata alla creazione di **gerarchie di classi** e grazie a essa si estende la possibilità di **riutilizzare componenti comuni** a più classi della stessa gerarchia.

Per mezzo dell'ereditarietà tra classi è possibile definire una **classe generale (classe base o superclasse)** avente la funzione di definire le caratteristiche comuni a uno specifico insieme di oggetti. Le caratteristiche della classe generale potranno quindi essere ereditate o estese da altre classi (**classi derivate o sottoclassi**) che integreranno le caratteristiche della classe base con elementi specifici. Le classi derivate, a loro volta, potranno configurarsi come superclassi per nuove classi, realizzando in questo modo la gerarchia.

Una classe derivata da una classe base mantiene le proprietà (**attributi**) e le operazioni (**metodi**) della classe base da cui eredita e a questi aggiunge i propri attributi e i propri metodi specifici.

La **classe padre** è quella che accomuna tutti gli oggetti di una stessa categoria. Qualsiasi figlio di questa classe avrebbe gli stessi metodi e le stesse proprietà, solo con le opportune modifiche. Ad esempio, alcuni oggetti possono avere caratteristiche differenti rispetto ad altri, pur appartenendo alla stessa categoria. In pratica la **superclasse** descrive il comportamento generale dei figli, non fa distinzioni iniziali e cerca di essere quanto più possibile imparziale per poter soddisfare tutti i requisiti dei figli.

Una **classe figlia** eredita tutti i metodi e attributi, in più può aggiungerne di nuovi e modificare quelli esistenti. Una classe figlia è una **specializzazione** della classe padre. Le classi derivate possono trovarsi sullo stesso livello della gerarchia perché svolgono lo stesso ruolo concettuale. Inoltre, una classe derivata può diventare a sua volta una nuova classe padre per altre sottoclassi, creando gruppi sempre più specifici di oggetti.

Il concetto di **polimorfismo** degli oggetti, tipico della programmazione **OO**, è strettamente collegato all'ereditarietà tra classi. Utilizzando il polimorfismo è possibile ottenere comportamenti e risultati diversi invocando gli stessi metodi a carico di oggetti diversi.

Mediante il polimorfismo è possibile trattare gli oggetti istanza delle classi di una gerarchia derivata da una classe base come se fossero istanze della classe base stessa.

Applicando il **polimorfismo** il codice è in grado di determinare il metodo da invocare in funzione della classe di cui l'oggetto è istanza.

Il funzionamento dell'ereditarietà e del polimorfismo nei linguaggi di programmazione **OO** prevede esplicitamente la ridefinizione nelle **sottoclassi derivate** dei metodi definiti in una **superclasse**. In alcuni casi l'implementazione di un metodo in una classe base diviene puramente formale.

Una **classe padre** può limitarsi a definire la firma di alcuni metodi (**interfaccia**) tralasciandone l'implementazione che riserva alle classi derivate. Metodi con questa caratteristica prendono il nome di **metodi astratti**.

Una classe in cui uno o più metodi sono astratti fattorizza le operazioni comuni a tutte le sottoclassi dichiarandone i metodi senza implementarli. A partire da una classe di questo tipo non è possibile istanziare oggetti, perché risulterebbero indefiniti rispetto all'eventuale invocazione dei metodi astratti.

Classi che presentano uno o più metodi astratti sono definite **classi astratte**: una classe astratta non viene definita allo scopo di istanziare oggetti, ma esclusivamente per derivare sottoclassi che specificheranno, implementandole, le operazioni dichiarate. Nei diagrammi **UML** una classe astratta viene indicata scrivendone la denominazione in carattere corsivo.